MacWave[1]的诞生

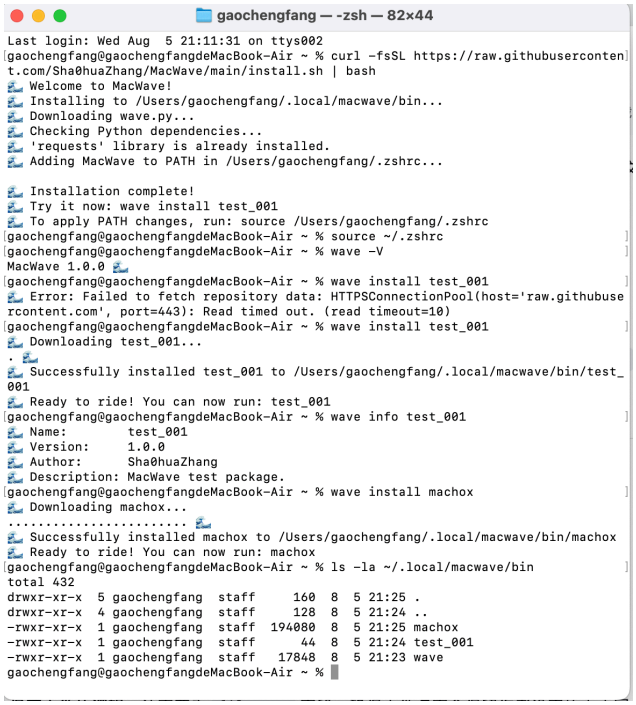
那是一次突发奇想。

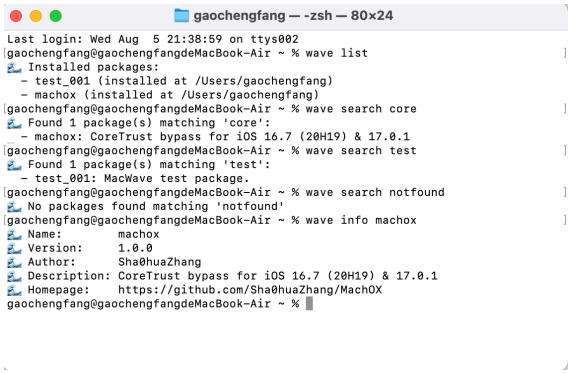
2026-08-05，我看着Homebrew[2]和MacPorts[3]，望着我越狱的iPhone[4]和安装了theos[5]、ldid(2.1.5-procursus7)[6]、TrollRestore[7]、Choma[8]的Mac，心里不禁想，为何没有一个专为越狱开发者而开发的包管理器呢？（作者注：那时候还不知道Homebrew已经收录了theos和ldid-procursus）

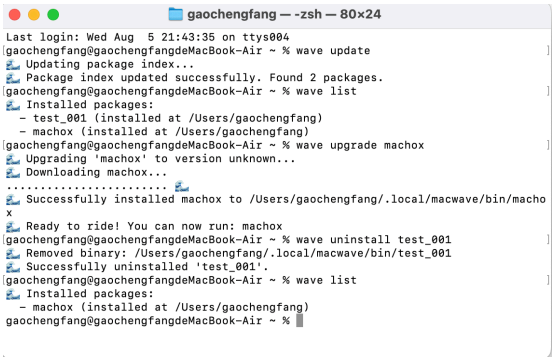
于是，我便要成为那个“Builder”，于是，这个“大作”就此拉开序幕！

当天，我登录好久都没用过的GitHub，创建了名为“MacWave”的项目。

为什么选择这个名字？其实是随便起的。最开始计划只是为了支持Mac，于是准备以Mac做前缀（名字对标MacPorts），随便起了一个名字。







当天，我便利用DeepSeek网页端（那时候还没配置VS Code）写出最小原型，技术栈选择了最熟悉的Python(3.14.8)，并在本机实现（见下图）。

我深知域名对于一个项目的重要性，于是第二天立刻就在NameSilo[9]购买了macwave.org作为项目域名（$14.99/年），并在DNS添加了一条302记录指向`https://github.com/Sha0huaZhang/MacWave`。

当时的目标只是做一个“自己能用的小程序”，并没有想太多，也完全没有想到日后的规模。

后来，我在几天内疯狂加功能，不过自己没参与架构的设计，导致后来被迫重构、自己开始设计核心架构和写逻辑，当然，这是后话。

然后我了解到Github Pages[10]，开始写网页，不再依赖302跳转。利用DeepSeek网页端写静态HTML，经过多次迭代、设计UI和图标，很快便写出来风格合格但屏幕适配不好的程序， 不过觉得“差不多得了，又不是造飞机大炮”，直到后来才优化。

此时的网页已经具有几乎所有的功能：最新版本、最新测试版（后被优化掉）、简介、命令、参数、联系方式……

我拥有了人生中第一个自定义邮箱：hi@macwave.org。

接着，我加入了多彩的进度条、帮助内容，终端不再那么枯燥；增加了强制的SHA256[11]验证，让下载变得安全；加入了`-C, --continue`和`.partial`，让下载变得无需担忧网络中断；在有风险的位置询问提示（用红色加粗字体），降低风险概率；下载先放到/tmp，不让不完整或危险的软件包污染其他部分；优化了多个提示，对用户友好……

终于，2026-08-16，经历大量迭代和多个beta、RC版、MacWave迎来首个正式版—1.0！

如果你足够细心，也会发现一些小变化：官网的下载命令由`curl | bash`改为`/bin/bash -c "$(curl …)"`，这是专为弱网环境（比如我家）设计的，防止流式的脚本因网络中断而下载一半。使用绝对路径是为了避免PATH[12]污染。

这里有一个重要的内容：由于当时是为了越狱设计的，精确的版本号十分重要。所以软件包全部被放在`~/.local/macwave/bin/{package}@{version}`，开始支持多版本隔离。

不过我渐渐发现，随着代码的增多，原本一个主程序越来越臃肿：`wave.py`从300多行直接增加到1200多行，不仅每次编辑十分困难，也包含了一些补丁————但他们不应该污染主程序。于是开始模块化（此处为自行设计），但涉及内容很多，于是准备先做出最小化验证。于是，在1.1版本，下载程序`pkginstaller.py`被先分离了出来。

进入2.0，大规模的解耦和模块化开始，这段时间，我进行了大量、大胆的技术尝试和试错，好在最终找到了不错的方案。

首先，我觉得arch字段的arm64、x86_64、Any太过混乱，于是分离为`pkginfo_arm64`和`pkginfo_amd64`，这便是今天软件包索引架构分离的原型。

由于1.0用的是`repo.json`。这种格式十分难以阅读且括号嵌套易出错，对于MacWave的包索引也不是很友好。我便进行了多个尝试。一开始，我使用DSL[28]和parser.rb去解析他们。这甚至迭代了好几次，但语法复杂、解析器发生过多次错误和坑，直到后来发现更好方案：

有一天，我想到，索引中大量重复的字段（包名、作者、……），能不能将他们只用一次呢？哦！做成目录不就行了！

于是我设立了infosource分支，专门存储软件包索引。在`https://raw.githubusercontent.com/Sha0huaZhang/MacWave/infosource/pkginfo_arm64`（或`amd64`）`/包名` 下，有下面几个文件：`_包名@版本号`（收录若干）、`_包名@common`。其中 `_包名@版本号` 记录软件包的URL和SHA256这些可能变化的东西，`_包名@common`记录不变的东西：作者、官网、……这样，软件包索引可以单独发release（无需用户更新，程序自动拉取），同时开启了独立的Index Update。

并且，不再使用本地索引————本地索引其实很麻烦，用户被迫下载了不必要的内容，且他们还需要手动更新索引（很多人会忘，比如我）————否则下载不到新版。而且开发者也需要发布索引更新……于是我改成了每次实时拉取。有一个否

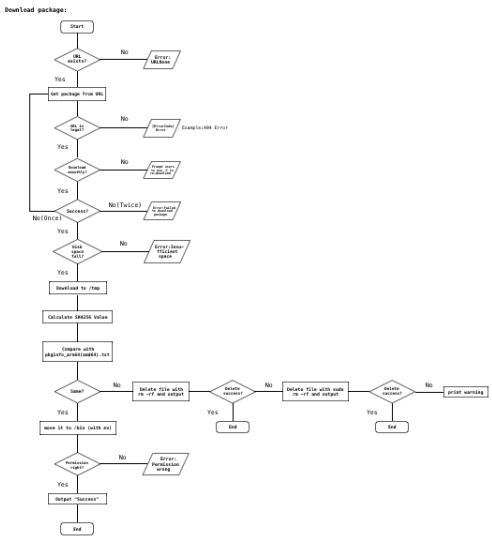
定的理由是用户需要联网才能查到软件包信息，但对于一个想下载软件包的人来说，联网是必须的，所以这不成问题。

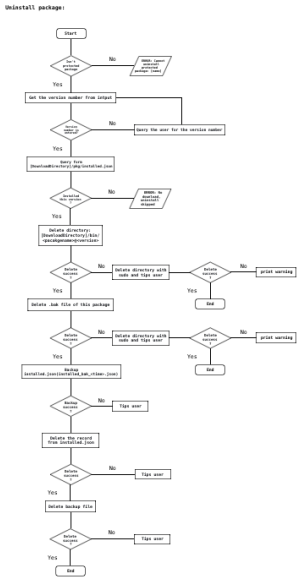
后来，我想到，很多操作其实用Shell比Python更容易，比如解压、移动、计算SHA256……，在Shell只需一条命令。于是解耦出很多`.sh`文件并保留一些用Python处理更方便的部分为`.py`文件。之后的目录结构和文件便和今天的MacWave十分相似。当时记下了几个基本原则：`lib/`放置核心程序，`pkg/`下放置和软件包有关的程序。

说起来不过几句，可2.0经历了大量失败、试错。途中，AI写的代码逻辑混乱，出现了一些怎么改也改不好的Bug，我想到，没有好的架构，就算再精妙的代码也难维护。于是做出了历史性的、极其重要的决定————重写整个MacWave，这决定了MacWave的今天和未来。

我去掉了主程序解析参数的功能，把它单独作为“分发入口”，建立映射表和字典，识别用户的命令，分发给原始字符串各个程序，让各个程序自行解析，这样一来，主程序仅有80行，非常简洁。

我对下载的每一个流程、每一步做什么都重新设计，画出了架构图，写出了核心逻辑，确保代码处于自己的掌控之下。





我开始尝试VS Code与DeepSeek API[13]，迎来先进的工具链，同样为MacWave的未来奠定了基础。这时还新增了规则， 写新的小版本时，同步创立版本号分支，持续与main同步，直到发布更新的小版本。

同时，我开始尝试本地Git[14]仓库代替纯网页端，这不仅是个人技术的进步，也是MacWave可持续发展的必要举措。

最终，2.0版本被跳过，2.1版本发布。这个版本没有任何功能性的更新，甚至还砍掉了一些非核心的功能————只是为了精简架构、便于日后维护。

我还发现以前的疏漏：忘记把软件包添加到PATH！这意味着用户必须使用绝对路径（`~/.local/macwave/bin/包名/版本号`），太痛苦了！于是立即修复。

我想到，如此精密的设计，怎能只服务于越狱开发？于是默默的把“jailbreak developers”都改为“software developers”，MacWave的目标从此转向作通用包管理器。同时也默默的把官网首页的“最新beta版”改为“最新版本发布日期”，因为似乎用户不需要beta版。

既然是通用包管理器，那用户就应该有更多的选择，而不是局限于特定的路径（如`/usr/local`[15], `/opt/homebrew`[16], `/opt/local`[17], `/sw`[18], `/opt/sw`[19], …）。我给出的选项有：1.`~/local/macwave` , 2.`/opt/macwave` , 3.`/usr/local/macwave`（仅Intel Mac）[20] 4.自定义。为了避免程序找不到目录，同时记录更多信息， 我把`config.json`（配置文件，记录路径等）和`VERSION.json`（记录版本信息）都放到了`/opt/macwave_config`。

以前wave本身视为一个软件包，像brew和port一样，在`bin/`（或对应目录）下，所以之前加了“受保护的软件包”：wave。不能`wave uninstall wave`，否则提示“受保护的软件包”。但在MacWave2.0，我想到更好的方案：把它们放到`lib/`, wave做成软链接。这样wave仍然可以调用，同时结构更合理，也无需引入“受保护的软件包”（2.0起取消此功能）。

在MacWave2.0，我还想到，未来应该同时支持Linux（因为Homebrew和Linuxbrew[21]也是这么做的），于是先预留了位置：LinuxWave仓库和官网linuxwave.macwave.org（后来在网站更新时被301重定向到新网址linuxwave.org)。不过这些预留到2.5版本才被启用。

随后我开始设计2.2，引入依赖处理。自然，依赖也按照了软件包一样的设计，严格区分版本号。不过一个难题出现，如何记录软件包的依赖和依赖的依赖呢？（否则删不干净或成了“依赖地狱”）传统软件（如MacPorts）一般通过数据库来记录，可是容易出错或丢失，那么有没有更好的方案呢？

别忘了以前我可是玩儿TrollStore[22]的，我也用过TrollStore安装Filza[23]，我看到TrollStore会在他的持久化助手（TrollStore PersistenceHelper[24]）的目录下放置一个隐藏标记文件文件（空）`.TrollStorePersistenceHelper`，在TrollStore安装的App下放置标记文件（空）`_TrollStore`，这方案不就来了吗！

经过大量的方案设计与考量、改良，得到的最终方案为：在软件包的目录下建立`_DEPS`文件，记录软件包使用了哪些依赖及版本，一行一个，用直引号""包裹；在依赖的目录下建立 `.depped_包名@版本号` 的空文件，记录依赖被哪些软件包所依赖。当然有些依赖也有依赖，同理。这本质上模糊了软件包和依赖的区别————确实，他们本质上其实都是可执行文件。

不过软件包和依赖还是有根本的区别，所以我选择了用`.depeed_pkg_包名@版本号`和`.depped_dep_依赖名@版本号` 来区分。

```
Last login: Sat Oct 24 23:50:54 on ttys011
gaochengfang@gaochengfangdeMacBook-Air ~ % cd ~/.local/macwave/bin
ls -d */ | head -n 5
fd@10.5.0/
fileicon@0.3.5/
fzf@0.74.4/
htop@3.5.3/
jq@1.8.2/
gaochengfang@gaochengfangdeMacBook-Air bin % cat wget@1.25.0/_DEPS
"gettext@0.21.0"
"libiconv@1.16"
"libidn2@2.3.8"
"libunistring@1.3"
"openssl@3.0.15"
"pcre2@10.42"
"zlib@1.2.13"
gaochengfang@gaochengfangdeMacBook-Air bin %
# 进入 openssl 的依赖目录
cd ~/.local/macwave/deps/openssl/openssl@3.0.15/

# 看看里面有没有隐藏的标记文件
ls -la | grep "depped"

-rw-r--r--   1 gaochengfang  staff    0  9 23 20:46 .depped_dep_libevent@2.1.13
-rw-r--r--@  1 gaochengfang  staff    0 10 12 16:36 .depped_pkg_wget@1.25.0
gaochengfang@gaochengfangdeMacBook-Air openssl@3.0.15 %
```

如何让软件包找到依赖呢？经过权衡，我选择了用Xcode Command Line Tools[25]修改Mach-O[26]寻址的方案。

中途我还设计了大量的命令族来允许开发者/用户去手动管理、替换依赖，但我想到：开发者/用户工具最重要的是省心，开发者/用户下班累的不行，哪有精力去手动一个个处理依赖？于是便引入了依赖自动管理：下载软件包先看看需要的依赖和版本是否存在，存在则直接记录，不存在则下载后记录。卸载软件包时看看本次卸载的软件包的依赖在卸载这个软件包后是否仍被其他软件包依赖。是则不会删除只是记录。否则删除并记录。依赖的依赖递归处理。

这个方案的优点有：1.原子性。由于标记文件是空文件，所以只存在“成功”“失败”和“有记录”“无记录”，不存在中间态，不会出现“记录一半”的尴尬情况。2.错误率低。利用空文件十分方便，下载时只需要创建文件而无需写文件，卸载时只需要删除和检测文件是否存在而无需读取文件。更不容易出错。3.便于迁移，不会因为忘记迁移数据库而导致丢失依赖记录。

于是————MacWave2.2发布！MacWave从此成为真正意义上的包管理器！

既然都做到这里了，那不兼容、不美观的官网显然不能满足我的要求，于是我重写官网到如今的样子，兼容各种屏幕尺寸和比例。

可是哪怕到了2.2版本，MacWave仍然没有自更新命令，于是我在2.3加入了自更新命令`selfupdate`。为什么不选择`update`？因为很多人区分不清`update`和`upgrade`（比如我），`selfupdate`直接表明自更新（update itself），同时对标MacPorts，是不错的选择。此后`selfupdate`经历多次迭代，如今已经支持从远程获取一些迁移命令而不是写死，防止架构发生变化时`selfupdate`集体失效。`selfupdate`的数据从`configdata`分支获取

随后是个小插曲————我发现`wave -h`居然还残留着1.x版本的命令，于是在2.3.1给修了。

我又发现：`wave -h`会打印所有的帮助信息，可如果用户只是不熟悉某个命令的用法，岂不是很麻烦？于是新增了`wave 命令 -h`，可以查看该命令的用法帮助，甚至有举例。

我想到，虽然PATH强制版本号（软件包@版本号）可以让用户知道并提醒他们使用的是哪个版本，但是这在日常并不方便，而且有时候用户只是想用个工具，不关心到底是哪个版本。于是我在MacWave2.4加入了link机制和一系列自动化处理。规则如下：

从 MacWave 2.4 起，建立不含版本号的软链接后，MacWave 会自动维护它指向的版本：

1.最高版本被卸载后，不含版本号的软链接会改指向剩余版本中的最高版本，被卸载版本自己的软链接一并撤销。

2. 安装会把不含版本号的软链接指向本次安装的版本；本次安装加上 --unlink 可跳过这一步。

3.某软件包不再有任何版本时，不含版本号的软链接会被撤销。

4.手动unlink过的软件包会被记住，之后的卸载不会重新为它建立链接。

5.无论何种情况，含版本号的名称始终可以调用，即使存在不含版本号的软链接。

还有如下的命令族：

```
wave link {pkg}
wave linkquery {pkg}
wave unlink {pkg}
wave link -a, all
wave install  {pkg}  --unlink
```

随后为之前安装的软件包设立了“一键link”命令：wave link -a（或—all）

然后我把官网的命令缩减成了一条：在安装命令后加入了 && source ~/.zshrc，防止有人忘记。

一次我卸载软件包：我发现卸载时要求选择版本号的设计在面对多个版本时十分方便，不过只有一个版本时有点显得“多此一举”，于是在2.4.2加入了只有一个版本时wave uninstall不再询问要卸载的版本号。

我想到，如果多个用户都使用了用户级安装，这个操作本该合法，可配置都写在/opt/macwave_config，会互相覆盖，不合适。于是我在2.5修改了配置文件的位置，规则如下：

配置文件不再固定放在/opt/macwave_config，而是跟着安装位置走：

• 安装到需要 sudo 的目录

（/opt/macwave、/usr/1oca1/macwave、家目录之外的自定义目录），配置在 /opt/macwave_config

• 安装到无需 sudo 的目录（~/.1oca1/macwave、家目录之内的自定义目录），配置在 ~/.config/macwave_config

程序读取时系统级优先：只要 /opt/macwave_config/config.json存在就使用它，否则回落~/.config/macwave_config，同时在install.sh和selfupdate做出了自动迁移

在这个版本，我把MacWave移植到了Linux，预留已久的LinuxWave终于得以启用。Mach-O寻址由Xcode Command Line Tools移植为ELF[27]，路径增加选项成为：1.~/.local/linuxwave, 2./opt/linuxwave, 3./usr/local/linuxwave, 4./home/linuxwave/.linuxwave, 5.自定义。配置文件位置分别为/etc/linuxwave_config和~/.cofig/linuxwave_config，共享安装（选项4）始终使用/etc/linuxwave_config，以保证全机所有用户解析到同一个安装。需要 patchelf、Python 3.14+，以及一次性提供sudo ; linuxwave账号不存在会自动创建。共享安装完成后，提示用户重新登录或执行newgrp linuxwave。这里我借鉴了Linuxbrew的用户组方案（感谢Linuxbrew，不过还是踩了不少坑），才有路径4（/home/linuxwave/.linuxwave），这确实是一个好方案，不仅能全局共享，还无需sudo即可安装软件包。

接着，我创建了史上最严苛的CI测试：采用随机化：各种命令或参数、错的对的、随机打断的等等。只要是你能想到的，CI测试都有可能发生。同时经过大量测试和优化，如今的CI已经十分稳定。

我想到，如果用户在脚本中使用了我的install.sh怎么办？脚本里可没人答Yy，于是在MacWave2.5，我新增了自动化安装：

安装

```
--silent, -S                不进行任何交互：目录菜单与许可协议都自动应答
--dir-option=N              直接选择菜单项 N（不弹出菜单）
--dir-option=N=DIR          选择菜单项 N；自定义项直接用 DIR 作为安装目录
--help                      列出全部选项
```

示例：

```
# 脚本已在本地
bash install.sh --silent --dir-option=1

# 直接从仓库安装
curl -fsSL https://raw.githubusercontent.com/MacWaveOrg/MacWave/HEAD/lib/
install.sh \ | bash -s -- --silent -dir-option=2
```

卸载

```
--force             跳过确认；删除 MacWave 及其配置
--remove-user       仅为与 LinuxWave 的参数兼容；MacWave 没有专用账号，实际不做任何事
--help              列出全部选项
```

在但我认为有一些不妥之处：1.既然选项3（Intel）或选项4（Apple Silicon）是自定义目录，那为什么还要写编号呢？这还容易出错；2.上述命令仍是`curl | bash`，应改为更稳妥的`/bin/bash -c "$(curl …)"`。于是针对这两项，发布了MacWave/LinuxWave3.0.1。

在MacWave/LinuxWave2.6，我意识到MacWave/LinuxWave已经是个体量巨大的项目，于是在2.6版本，就像mxcl[2]把Homebrew仓库迁移给Homebrew组织一样，我将个人名下的MacWave/LinuxWave迁移给了MacWaveOrg/LinuxWaveOrg组织，新购入linuxwave.org（旧域名linuxwave.macwave.org被301到新域名），专项组织只经营一个项目。这是一次史无前例的大迁移，为了便于维护，一些分支被迁移为仓库。旧仓库的只读快照被复制并存档，在一周后这些，我在这些分支的名字前面添加了legacy-前缀。

由于新域名linuxwave.org的购入，LinuxWave专门发布了2.6.2版本。

我想到，既然已经有专用组织，infosource、configdata、gh-pages完全可以做成仓库，这样还更方便，因此发布了MacWave/LinuxWave3.0，同时给旧版本用户7天的迁移期限。

MacWave/LinuxWave2.6-3.0版本，总共发生了以下的大迁移：

迁移项	迁移前	迁移后	迁移版本	备注
MacWave仓库	https://github.com/Sha0huaZhang/MacWave	https://github.com/MacWaveOrg/MacWave	MacWave2.6	仓库地址迁移
LinuxWave仓库	https://github.com/Sha0huaZhang/LinuxWave	https://github.com/LinuxWaveOrg/LinuxWave	LinuxWave2.6	仓库地址迁移
MacWave旧仓库只读快照		https://github.com/MacWaveOrg/legacy-MacWave	MacWave2.6	仓库只读快照
LinuxWave旧仓库只读快照		https://github.com/MacWaveOrg/legacy-LinuxWave	LinuxWave2.6	仓库只读快照
MacWave/infosource分支	https://github.com/Sha0huaZhang/MacWave/tree/infosource	https://github.com/MacWaveOrg/infosource	MacWave3.0	分支独立为仓库，随后更名为legacy-infosource
LinuxWave/infosource分支	https://github.com/Sha0huaZhang/LinuxWave/tree/infosource	https://github.com/LinuxWaveOrg/infosource	LinuxWave3.0	分支独立为仓库，随后更名为legacy-infosource
MacWave/gh-pages分支	https://github.com/Sha0huaZhang/MacWave/tree/gh-pages	https://github.com/MacWaveOrg/Pages	MacWave3.0	分支独立为仓库
LinuxWave/gh-pages分支	https://github.com/Sha0huaZhang/LinuxWave/tree/gh-pages	https://github.com/LinuxWaveOrg/Pages	LinuxWave3.0	分支独立为仓库

迁移项	迁移前	迁移后	迁移版本	备注
MacWave/configdata分支	https://github.com/Sha0huaZhang/MacWave/tree/configdata	https://github.com/MacWaveOrg/configdata	MacWave3.0	分支独立为仓库，随后更名为legacy-configdata
LinuxWave/configdata分支	https://github.com/Sha0huaZhang/LinuxWave/tree/configdata	https://github.com/LinuxWaveOrg/configdata	LinuxWave3.0	分支独立为仓库，随后更名为legacy-configdata
LinuxWave官网	linuxwave.macwave.org	linuxwave.org	LinuxWave2.6.2	官网独立为主域名

在MacWave3.0.2，我作出了一个便利的小修改：--ver从此可以用=连接版本号（此前只能用空格），现在安装特定版本的软件包有三种方式。以ffmpeg为例，有下面三种：

```
wave install ffmpeg@9.0
wave install ffmpeg –ver 9.0
wave install ffmpeg –ver=9.0
```

在MacWave3.0.2，我作出了一个便利的小修改：--ver从此可以用=连接版本号（此前只

后来，Python3.15发布，为了尝鲜新特性同时兼容更高版本，我便升级了我的Python（3.14.8 >>> 3.15.0），结果MacWave的`linkquery`崩了！提示缺少`request`库，我发现，MacWave的Shebang为 `#!/usr/bin/env python3` ， 此时env变为3.15，可是request装在了pyenv的3.14上，于是参考Homebrew打包Ruby的做法，我在3.1起打包了MacWave自己的、独立的CPython解释器，这样一来，不仅避免了用户有过高/过低（很多时候不便更新/降级）的Python版本的问题，甚至都无需用户安装Python，这无疑极大的降低了使用门槛。

——————————— 编辑于 2026-10-15，此时最新版：3.1.0 ———————————

任何一个伟大的工具，都有一个微不足道的开始。

角标释义：

1.MacWave：由Sha0huaZhang在2026-08-05的开发的包管理器，于2026-08-16发布首个正式版，现已迁移至MacWave组织名下，官网为macwave.org。

2.Homebrew：由Max Howell在2019年开发的包管理器，特点为下载预编译的二进制包，无需本机源码编译。现已从个人名下移交至Homebrew组织名下，官网为brew.sh。

3.MacPorts：由几位苹果公司员工在 2002 年共同发起，最初作为 OpenDarwin 项目的一部分启动的包管理器。特点是将软件安装到独立的目录下，避免与系统自带软件或其它包管理器的文件混在一起，减少冲突风险，官网为macports.org。

4.作者曾经的iPhone主力机，iPhone XR，白色，128GB，iOS15.3.1（19D52），曾在此使用TrollStore、Dopamine。

5.Theos：由Dustin Howett开发的一个用于 iOS 越狱插件开发的开源跨平台构建系统，提供简化代码编写、编译和 .deb 打包流程等功能，现已转为社区维护。

6.ldid：Jay Freeman（Saurik）开发的一个轻量级命令行工具，用于查看和修改 iOS/macOS Mach-O 二进制文件的签名与授权信息，现已转为社区维护，使用最广泛的版本为ProcursusTeam维护版。

7.TrollRestore：由JJTech（@JJTech0130）主导开发的、专为iOS16.7RC和17.0开发的TrollStore电脑端安装工具，也可用于最低iOS15.2安装TrollStore。

8.ChOma：由Lars Fröder （@opa334）开发的一个C语言库，用于构造CVE-2023-41991签名漏洞存在的恶意签名，主要被使用在TrollStore上。

9.NameSilo：一家全球知名的 ICANN 认证域名注册商。

10.GitHub Pages：GitHub 提供的免费静态网站托管服务，直接从仓库读取文件并自动发布为网站。多用于托管个人简历、项目文档或博客，无需自购服务器，公共仓库用免费账户即可使用。

11.SHA256：种密码学哈希函数，能把任意长度的数据压缩成固定256位（32字节）的独特“指纹”，且几乎无法从指纹反推原数据或找到两个相同指纹的不同数据。

12.PATH：操作系统用来查找可执行文件的环境变量，决定了在命令行输入命令时系统会去哪些目录里找对应的程序。

13.DeepSeek API在VS Code的插件，价格低廉、便于使用。

14.本地Git：在自己的电脑上运行的Git版本控制系统，用于跟踪代码修改、管理提交历史，无需联网即可完成提交、分支、回滚等操作。

15.`/usr/local`：Homebrew在Intel Mac的安装路径。

16.`/opt/homebrew`：Homebrew在Apple Silicon Mac的安装路径。

17. `/opt/local`：MacPorts的安装路径。

18. `/sw`：Fink的安装路径。

19.`/opt/sw`：Fink在macOS Catalina10.15后的安装路径，此时系统根目录（`/`）变为只读，`/sw`无法创建。

20.`/usr/local`：苹果在Apple Silicon Mac开始向`/usr/local`写入文件（`/usr/local`不再为空），同时不推荐写入此目录。

21.Linuxbrew：Homebrew最初不支持Linux，社区在2012年将其移植为Linuxbrew。核心程序在2019年发布的Homebrew2.0.0合并回Homebrew，Homebrew-core在2021年完成全部合并。

22.TrollStore：iOS 上的“永久签名”工具，可安装未经 App Store 签名的应用，利用签名漏洞让应用长期有效，可用于iOS14.0beta2-iOS16.7RC/17.0。

23.Filza：越狱/TrollStore 环境常用的iOS 上的文件管理器，可读写系统文件（仅有根越狱可写）。

24.TrollStore PersistenceHelper：TrollStore 的“持久化助手”，用于在系统更新或重启后维持 TrollStore 及其安装的应用可用。

25.Xcode Command Line Tools：苹果提供的命令行开发工具集，包含编译器、`codesign`、`install_name_tool` 等，文中用于修改 Mach-O 的依赖寻址。

26.Mach-O：macOS/iOS 的可执行文件格式，记录二进制代码、依赖库路径、签名等信息。

27.ELF：Linux 的可执行文件格式，相当于 macOS 的 Mach-O；文中指 LinuxWave 在 Linux 上做依赖寻址时对应的二进制格式。

28.DSL：领域特定语言，这里是为描述软件包索引而设计的小型专用语法。